Ocean Engineering 362 (2026) 126356

Contents lists available at ScienceDirect

Ocean Engineering

journal homepage: www.elsevier.com/locate/oceaneng

Research paper

Regulatory requirement classification and semantic verification for model-based ship design

Haitham Al-Shami *, Riku Ala-Laurinaho, Raine Viitala

Aalto University, School of Engineering, Department of Energy and Mechanical Engineering, 02150 Espoo, Finland

ARTICLE INFO

Keywords:
Ontology-based validation
Model-based engineering
Maritime engineering
Semantic web technologies

ABSTRACT

Maritime regulations define constraints on ship design spanning areas such as structural safety, environmental emissions, and lifecycle requirements. This paper proposes a method for classifying regulatory requirements by verification type and encoding them as machine-executable constraints over a semantic model built with the Web Ontology Language (OWL), the Shapes Constraint Language (SHACL) and SPARQL. Each regulatory clause is assigned to one of five categories — Static, Static Calculation, Dynamic, Complex, and Physical Test — based on their verification type. SHACL shapes for directly verifiable categories validate parameter values and computed quantities against regulatory constraints. For regulations requiring simulation-driven verification, the model is first validated for the variables and component relationships the simulation requires before the simulation can proceed. Physical verification processes are tracked by explicit status in the same semantic model. To validate the approach, 201 requirements from the Finnish-Swedish Ice Class Rules published by TRAFICOM are formalized, and verification against independently published sample ship data resolves 50 requirements to passed status, while the remaining 151 are traceable to specific design parameters not yet available in the model. The approach transforms the regulatory text into active constraints on the design model, providing engineers with reliable compliance management as the regulatory complexity grows.

1. Introduction

Engineering design in safety-critical domains operates under extensive regulatory constraints. The maritime industry is governed by one of the most comprehensive international regulatory frameworks of any engineering sector (Sampson et al., 2014). Because of this, ships need to satisfy a wide range of international and national regulatory rules that cover safety, environmental performance, and operational constraints (Valdivieso, 2019; Wei and Liu, 2022). Regulatory verification in maritime design is traditionally a manual and document-centered process, where compliance with every applicable rule is difficult to guarantee as design data and regulations evolve. Moreover, the risk of discovering non-compliance during the production phase remains significant, and such findings can result in redesign, schedule delays, and higher project costs (Sampson et al., 2014). Recent advances in Model-Based Systems Engineering (MBSE), co-simulation, collaborative design frameworks, formal requirement specification, and semantic technologies create an opportunity to shift regulatory verification into earlier stages of the ship design lifecycle. Using these advances, when regulatory constraints are linked to models from the start of the development process, a larger share of compliance evidence can be obtained through systematic, repeatable verification, and the risk of non-compliance is reduced through improved traceability and reuse of formalized regulatory knowledge across related ship designs.

Ships represent substantial capital investments with development cycles that span years with steep manufacturing costs. These high costs are why Physical prototyping and testing can be substantially expensive. Furthermore, production volumes for larger ships are typically low, making each design iteration costly. When compliance violations are discovered during classification society review, sea trials, or during operational service, the resulting redesigns, schedule delays, and potential retrofits impose enormous financial and operational burdens (Alblas and Pruijn, 2024; Gourdon, 2019). These pressures are driving the maritime industry toward digital verification methods that move compliance verification earlier in the design spiral, where corrections remain more flexible and less costly.

In recent years, ship design has become subject to increasingly complex regulatory frameworks as international, regional, and national requirements are added to existing rules. Det Norske Veritas (DNV) describes maritime regulation as a changing landscape shaped by global and regional frameworks, including measures of decarbonization of

* Corresponding author.
E-mail address: haitham.al-shami@aalto.fi (H. Al-Shami).

https://doi.org/10.1016/j.oceaneng.2026.126356
Received 2 January 2026; Received in revised form 21 May 2026; Accepted 1 June 2026
Available online 11 June 2026

the International Maritime Organization (IMO), EU ETS (European Union Emissions Trading System), and FuelEU Maritime (DNV, 2025). Lloyd's Register similarly reports that a broad set of new IMO and International Labour Organization (ILO) requirements is entering into force, and notes that the number and complexity of regulatory updates are increasingly difficult for industry stakeholders to interpret (Lloyd's Register, 2025). Therefore, shipyards must address the requirements of the IMO, regional requirements of the European Union (EU), and specific requirements of the national or operating area such as the Finnish-Swedish Ice Class Rules issued by TRAFICOM (Finnish Transport and Communications Agency (Traficom), 2024; Finnish Board of Navigation, 1985; European Parliament and Council, 2023b,a, 2015; International Maritime Organization, 2021). New requirements related to carbon pricing, carbon-intensity reporting, and ballast water management add further compliance obligations without replacing established safety and pollution-prevention baselines such as SOLAS (Safety of Life at Sea) and MARPOL (International Convention for the Prevention of Pollution from Ships). This cumulative regulatory setting makes compliance difficult to manage through manual document reviews alone, as shipyards need to maintain consistency across multiple authorities, design domains, and verification methods. Due to this accumulation of regulations, compliance is a continuing engineering constraint that requires traceable links between regulatory clauses, design parameters, calculations, simulations, and physical test evidence.

Another aspect of ship design that compounds the challenge of regulatory compliance is the collaborative nature of modern ship design and manufacturing. Shipyards integrate components from numerous suppliers, and compliance applies at both the component and system level. For example, the TRAFICOM ice class rules (Finnish Transport and Communications Agency (Traficom), 2024) require that an individual engine deliver at minimum 2800 kW for an IA Super vessel, which is a constraint on a single component from one supplier. However, the overall minimum propulsion power P_{min} depends on the geometry of the hull, the diameter of the propeller, and the resistance to ice, which are parameters that come from different suppliers and engineering teams. A compliance failure in the integrated calculation may not be traceable to any single component in isolation and often requires further analysis. This distributed development process creates significant coordination challenges for regulatory compliance verification. The process of collecting component specifications from multiple vendors and manually verifying that the integrated system satisfies all applicable regulations is error-prone and time-consuming (Sampson et al., 2014).

To address these challenges, the maritime industry increasingly relies on virtual verification during early design stages. This is in part enabled by Model-Based Systems Engineering (MBSE), which structures the design process around interconnected component models that can be queried, analyzed, and shared between engineering teams (Gholami et al., 2020). These models serve as the basis for computational verification methods such as Finite Element Methods (FEM) for structural analysis, Computational Fluid Dynamics (CFD) for hydrodynamic performance, and co-simulation methods for integrated multi-domain analysis (Ala-Laurinaho et al., 2024). The trend in MBSE is towards performing an increasing share of Verification and Validation (V&V) activities through these mature methods, which reduces reliance on late-stage physical testing and allows compliance evidence to be generated earlier in the design lifecycle (Serester et al., 2015).

Classification societies such as DNV, Bureau Veritas, and Lloyd's Register initially verify that ship designs satisfy applicable rules before construction, and their plan approval processes increasingly rely on structured digital design information. Recent initiatives such as the Open Class 3D Exchange (OCX) standard address this challenge from the perspective of data-exchange by linking 3D design models to classification society rule-checking platforms (Astrup et al., 2022). This direction reduces ambiguity in the submitted design data and supports more automated approval processes as the OCX exchanged serves as a standard format. However, in this case, compliance is treated primarily as a process of exchanging design information for external validation after the initial design and parameters of the ship have been completed. The complementary problem addressed in this paper is earlier in the design phase, particularly in how regulatory knowledge can operate as an active constraint on evolving ship design data before a complete approval package is assembled. This issue requires that regulatory clauses be represented as machine-readable rules, linked to design variables, and organized by the type of evidence needed for verification. Consequently, the core problem addressed in this paper is the lack of a formal regulatory layer that gives shipyards and suppliers continuous feedback on compliance status and verification readiness during model-based ship design.

Therefore, this paper proposes a semantic validation framework that represents ship design data as a Resource Description Framework (RDF) and Web Ontology Language (OWL) graph and formalizes regulatory requirements using semantic tools, namely the Shapes Constraint Language (SHACL) and the SPARQL Protocol and RDF Query Language (SPARQL) validation rules. This approach is motivated by three properties of semantic web technologies that fit regulatory compliance well: regulatory rules can be expressed as machine-executable constraints that run automatically against the design data, missing parameters are flagged as non-compliance rather than silently ignored, and new regulations can be added as independent rule sets without modifying the underlying design model. Regulatory clauses are classified into categories that correspond to different verification methods, including direct parameter checks, formula-based calculations, simulation-dependent evaluations, and physical inspections. This classification determines how each clause is encoded, either as an SHACL constraint that evaluates the requirement directly on the model or as a simulation readiness assurance that confirms that the model contains all necessary parameters for external verification. The proposed pipeline is illustrated in Fig. 1, which shows the process of how the design process and regulatory compliance are bridged together.

The objective of this paper is to determine whether regulatory requirements from safety-critical engineering domains in maritime ship design can be systematically classified by verification type and encoded as machine-executable constraints over a semantic system model. The Finnish-Swedish Ice Class Rules published by TRAFICOM (Finnish Transport and Communications Agency (Traficom), 2021) serve as the evaluation case because they span structural, propulsion, operational, and physical inspection requirements in all major aspects of ship design, and therefore exercise all five categories of the proposed classification. To complete this evaluation, 201 requirements from TRAFICOM regulation are extracted, classified, and formalized as SHACL shapes, and the resulting constraint set is verified against independently published sample ship data from TRAFICOM's verification annex.

The main contributions of this work are:

1. Introduce a Model-Based methodology that integrates regulatory requirements into simulation-driven ship design. The methodology uses a semantic system model together with SHACL and SPARQL validation to bring regulatory compliance into early design iterations, link clauses to specific components and variables, and treat simulation readiness for dynamic and complex requirements as an explicit validation category.
2. Propose a generic regulatory layer on top of the ReqTLo (Al-Shami et al., 2025) ontology that represents individual regulation clauses as `RegulatoryRequirement` instances, classified into five requirement types (Static, Static Calculation, Dynamic, Complex and Physical Test), and linked to concrete components and variables in a model-based ship description.
3. Validate the classification and encoding approach against 201 requirements of the TRAFICOM Finnish-Swedish Ice Class Rules, and verify the resulting constraints against independently published sample ship data to demonstrate end-to-end compliance checking with traceable outcomes.

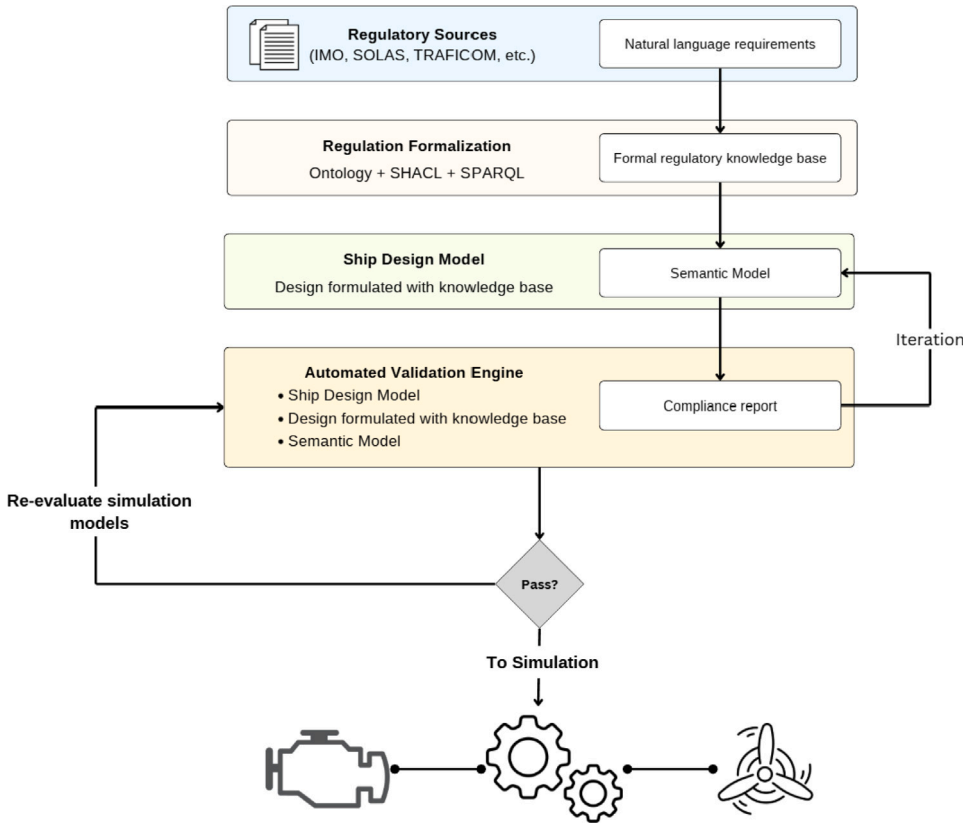

Fig. 1. The overall pipeline of the approach outlined in this paper from regulatory text to their inclusion in the design process of the ship.

2. Background

This section details background on the semantic technologies used in this work, the role of the Requirements Temporal Logic ontology (ReqTLo) in system design, and existing approaches to representing regulations in machine-readable form.

2.1. Semantic technology background

Maritime regulations are written in natural language for human interpretation, and computers cannot directly check natural language text against design data. As regulations grow in number, change more frequently, and need to be re-evaluated across many design iterations, manual interpretation no longer scales. To automate compliance check, regulations need to be converted into a structured machine-readable form that is formalized so that they can be automatically evaluated against design data (Zhang and El-Gohary, 2017). Recent advances in Natural Language Processing (NLP) support this transformation at scale (Zhang and El-Gohary, 2015), and the resulting structured rules form the basis of Automated Compliance Checking (ACC) systems.

ACC can be interpreted over two assumptions, the Closed-World Assumption (CWA) or an Open-World Assumption (OWA), each resulting in distinct formalisms. Under the CWA, information that cannot be proven compliant is treated as non-compliant (Minker, 1982). Under the OWA, the absence of information is interpreted as unknown, and non-compliance must be demonstrated explicitly (Moore and Van Pham, 2015) over known knowledge. For example, consider a ship model that does not include an engine power value. Under the CWA, this is treated as non-compliance with the 2800 kW minimum for an IA Super vessel, since the requirement cannot be proven satisfied. Under the OWA, the same model returns an unknown verdict because the absence of data is not evidence of violation. For engineering domains with safety-critical constraints, the closed-world view is often preferred, since missing or incomplete information should be treated as a potential violation. This choice has a direct impact on the selection of semantic technologies for regulatory modeling.

Semantic web standards provide mechanisms for representing domain knowledge and validating design data. The Resource Description Framework (RDF) is used to represent data as subject–predicate–object triples, and the Web Ontology Language (OWL) builds on RDF to define concepts, relationships and axioms in a shared vocabulary (Antoniou and Harmelen, 2009). OWL supports rich classification and reasoning over domain concepts, but it operates under the open-world assumption, which limits its usefulness for deterministic compliance checking, where missing information needs to be flagged for violation.

The Shapes Constraint Language (SHACL) is another semantic web language that operates over closed-world semantics for the validation of RDF graphs. SHACL defines constraint shapes that can be applied to RDF data to check conformance against explicit rules (Heimsbakk and Torkelsen, 2023). SHACL validation produces detailed reports that identify where a graph violates the given constraints, which is important in safety-critical domains such as ship design where missing parameters or incorrect values are reported as non-compliance (Zhong et al., 2012).

SHACL constraint rules can capture regulatory logic at different levels (Nuyts et al., 2023):

- Value constraints, which verify that properties satisfy threshold or range conditions.
- Relational constraints, which ensure consistency among related design entities (for example, that all required components are present).
- Mathematical constraints, which evaluate formula-based rules using SPARQL extensions, since SHACL-Core has limited arithmetic support.

In this way, OWL and SHACL are complementary. OWL encodes domain concepts and relationships in a reusable vocabulary (Antoniou

and Harmelen, 2009; Liu and Cheng, 2024), while SHACL provides deterministic validation of completeness and conformance under a closed-world assumption (Heimsbakk and Torkelsen, 2023). Together, they offer a formal, machine-interpretable framework that is suitable for automated regulatory compliance validation.

2.2. System design in ReqTLo

Ship design has a collaborative aspect, with OEMs supplying components to shipyards or subsystem integrators that conduct analysis over the integrated system. One way to structure this collaboration is through Model-Driven Engineering (MDE), in which the system specifications, requirement tests, and dynamic models are represented as interconnected models of a single shared system description. This shared model supports collaborative design across the organizations involved and provides a basis from which simulation models can be generated. In simulation-driven ship design, these models are used to evaluate performance, compare design alternatives, and assess compliance with requirements. To support automated reasoning over them, the system structure and requirements should be represented in a standard, machine-readable form.

In previous work, the Requirements Temporal Logic ontology (ReqTLo) was developed to formalize dynamic requirements for simulation-driven design (Al-Shami et al., 2025). ReqTLo provides a standardized representation of systems and their requirements as RDF graphs, with components, variables, and time-dependent behaviors modeled through a combination of OWL and supporting domain ontologies.

For maritime systems, this work imports three supporting ontologies into ReqTLo. Components and their observable properties are modeled using the Semantic Sensor Network (SSN) ontology and its companion Sensors, Observations, Samples and Actuators (SOSA) ontology, which provide domain-neutral representations and replace the road–vehicle ontology used in the original ReqTLo implementation. System variables such as engine power or ice resistance are represented as `sosa:ObservableProperty` instances linked to their parent components through `ssn:isPropertyOf`. Physical units are represented using the Quantities, Units, Dimensions, and Types (QUDT) ontology, and time-dependent requirements are expressed through an ontology for Metric Temporal Logic (MTL).

Within this framework, a ship is represented as an interconnected set of components, variables and temporal requirements. The ReqTLo model serves as the semantic description of the system that can be queried in OWL and then validated and linked to analysis models. This paper reuses ReqTLo as the base system model and adds a generic regulatory layer on top of it. The regulatory layer introduces a classification of regulatory requirements into five categories, a set of SHACL and SPARQL rule patterns that operate on ReqTLo instances, and a mechanism for recording the verification status of individual requirements. These additions are independent of any specific regulatory domain and could, in principle, be applied to other regulated systems. The ice class rules are used as a running example to demonstrate how this combination of ReqTLo and the regulatory layer supports automated compliance-oriented checks and simulation readiness analysis.

3. State of the art

This section reviews existing approaches for representing regulations and engineering requirements in machine-interpretable form. The state of the art first considers semantic technologies because they provide the main mechanism used in this work to connect regulatory clauses with design data and validation rules. Then, this direction is compared with model-based systems engineering approaches that represent requirements inside system models, especially SysML methods. This structure separates two related questions. The first is how regulatory knowledge can be formalized as executable constraints, and the second is how those constraints can be connected to engineering models used during ship design.

3.1. Regulation representation in semantic technology

Regulations impose constraints on ship designs at multiple levels, including emission standards (Ritari et al., 2023), domain-specific requirements such as ice class regulations (Finnish Transport and Communications Agency (Traficom), 2021), and general safety standards (Tadros et al., 2023). Shipyards need to satisfy these regulatory constraints while at the same time meeting customer-specific requirements and operational objectives. For maritime applications, the evolving complexity of environmental and safety regulations makes manual compliance management increasingly difficult.

Automated Compliance Checking (ACC) relies on representing regulatory requirements in machine-readable formats. Semantic-based approaches encode regulations as logical rules over an ontology, which allows automated reasoning and consistency verification (Zhang and El-Gohary, 2017). Natural Language Processing (NLP) techniques can support this process by extracting candidate requirements and entities from regulatory text and transforming them into structured representations (Zhang and El-Gohary, 2015). These structured representations can then be mapped to domain ontologies and used as the basis for semantic validation.

In the maritime domain, recent work has examined the effectiveness of environmental regulations and highlighted a gap between the intention of the regulatory and its implementation in practice (Olaniyi et al., 2024). Industry stakeholders report that frequent regulatory changes, vague formulations, and ambiguous timelines lead to design uncertainty and repeated adaptation of technical solutions. These findings indicate a need for more agile and digitally supported frameworks for regulatory compliance.

Several approaches for automated ACC have been proposed in other domains. One area of research uses formal logics such as Linear Temporal Logic (LTL) to express regulations as properties that can be verified against system models. Elgammal et al. propose a pattern-based visual language on top of LTL that helps compliance experts specify requirements for business process models (Elgammal et al., 2016), and Sannier et al. extract normative structures from regulatory documents to support automated conformance checking (Sannier et al., 2011). Another area of research uses semantic web technologies such as OWL and SHACL. In this approach, regulations are encoded as constraints over RDF graphs that represent system designs. OWL defines the vocabulary and relationships of the domain, SPARQL provides flexible querying over both design data and regulatory knowledge, and SHACL enforces concrete validation rules such as threshold conditions, relational dependencies, and mathematical expressions (Heimsbakk and Torkelsen, 2023; Zhong et al., 2012; Nuyts et al., 2023). Under the closed-world assumption, missing or inconsistent information is treated as non-compliant, which fits the needs of safety-critical domains where incomplete data should evaluate to a false model.

Within semantic web approaches, SHACL-SPARQL constraints can cover a wide range of regulatory logic, but some engineering formulas involve operations such as fractional exponents or iterative numerical procedures that exceed SPARQL's native arithmetic. To overcome this limitation of SPARQL, the SHACL JavaScript Extensions specification (SHACL-JS) (Knublauch and Maria, 2017) allows arbitrary JavaScript functions to be embedded in SHACL shapes. However, SHACL-JS never advanced beyond a W3C Working Group Note and did not undergo the full standardization process that produced the SHACL Core and SHACL-SPARQL Recommendations. The tooling trajectory reflects this status. The original reference implementation (TopQuadrant's `shacl-js`) (TopQuadrant, 2017) is no longer actively maintained, and Zazuko's rdf-validate-shacl (Zazuko, 2025), which began as a fork of that library, has since removed SHACL-JS support entirely to focus on SHACL Core. In contrast, SHACL Core and SHACL-SPARQL continue to receive active standards work in SHACL 1.2 (Knublauch et al., 2026) and are implemented by widely used processors such as Apache Jena, Eclipse RDF4J, pySHACL, and shacl-engine (with its SPARQL plugin).

Table 1
Theoretical comparison between standard SysML v2 metamodel assumptions and the proposed semantic (SHACL) approach for regulatory compliance.

Dimension	SysML v2 (Type-Theoretical)	Proposed (Graph-Based/SHACL)
Constraint location	**Intrinsic:** Constraints are defined within the Block/Type definition or bound via explicit satisfaction relationships.	**Extrinsic:** Constraints are defined as external shapes that overlay the data graph; the model structure remains agnostic to the rule set.
Model validity	**Binary (Well-formedness):** An instance that violates a multiplicity constraint (e.g., missing parameter) is typically considered structurally ill-formed.	**Contextual (Conformance):** A graph with missing data remains a valid RDF graph; it simply fails a specific "Readiness Shape," allowing for graceful error handling.
Coupling	**Tight Coupling:** Regulatory updates often require modifying the system hierarchy or requirement satisfaction links.	**Structural Decoupling:** New regulations can be applied as new validation overlays without altering the underlying system ontology.
Epistemology	**Prescriptive:** The model dictates what the instance *must be* to exist.	**Descriptive:** The shapes describe patterns to *look for* in existing data.

Beyond tooling, keeping the complex numerical computations outside the semantic validation layer avoids reimplementing such logic inside SHACL shapes which would duplicate code that exists in trusted tools, introduce a second implementation requiring its own verification, and couple the regulatory validation layer to specific numerical procedures. The approach adopted in this work therefore uses SHACL-SPARQL for directly checkable constraints and treats the remaining formulas as simulation-readiness targets, keeping the semantic layer responsible for structural completeness and status tracking and delegating numerical computation to external solvers.

Existing work on ontology-based regulatory compliance largely assumes that regulations can be verified directly against a static system description. Little attention has been paid to how regulatory rules interact with simulations, or to the question of which parts of a regulation can realistically be checked by semantic constraints and which should be delegated to simulation models or physical tests. In particular, prior approaches do not treat simulation readiness itself as an explicit validation target, that is, they do not validate whether a system model contains all variables and structures needed to generate compliance evidence through simulation. This work addresses that gap by combining ReqTLo with SHACL in a way that both encodes directly verifiable rules and systematically verifies that the ReqTLo model is complete enough to support simulation-based verification for the remaining dynamic and complex requirements.

3.2. Other regulatory formulation methods

Current Model-Based Systems Engineering (MBSE) methodologies primarily employ the Systems Modeling Language (SysML) to capture requirements. Approaches such as Model-Based Structured Requirements attempt to formalize compliance by mapping textual regulations into SysML requirement blocks (Herber et al., 2022). SysML Parametric Diagrams can bind system properties to mathematical constraints. This is effective for physics equations; however, approaches using SysML v1 and SysML v2 models are generally designed under assumptions of completeness relative to their metamodel where a missing mandatory parameter results in a model instance being structurally invalid or incorrectly specified. This binary validity state makes it difficult to perform verification during early design phases where data is still sparse.

SysML v2 introduces a formal metamodel (KerML) that supports constraint definition. Constraints in KerML are typically intrinsic to system block definitions or explicitly bound through satisfaction relationships (Jansen et al., 2022). This tight coupling between the system architecture and the regulatory framework makes it difficult to reuse constraint sets across different designs or to update regulatory rules independently of the system model. In contrast, the proposed semantic approach utilizes Structural Decoupling. In SHACL, regulatory constraints are treated as external shapes that overlay the data graph. This allows the same design model to be validated against multiple, conflicting, or evolving regulatory schemas without necessitating changes to the underlying system ontology, a flexibility that is theoretically cumbersome to achieve in rigid type hierarchies. Table 1 illustrates the main differences between SysML and semantic approaches.

To illustrate the practical difference in the Model Validity row of Table 1, consider the minimum engine power requirement for an IA Super vessel. In SysML v2, this requirement can be expressed as a constraint definition with a satisfaction relationship to the ship model:

```
private import ScalarValues::*;

constraint def MinEnginePower {
    in enginePower : Real;
    enginePower >= 2800.0
}

part def Engine {
    attribute power : Real;
}

requirement def MinEnginePowerReq {
    subject ship : IceShip;
    require constraint : MinEnginePower {
        in enginePower = ship.engine.power;
    }
}

part def IceShip {
    attribute iceClass;
    part engine : Engine;
    satisfy requirement : MinEnginePowerReq;
}
```

Listing 1: SysML v2 minimum engine power constraint.

If `engine.power` has not been assigned a value during early design, the binding `in enginePower = ship.engine.power` cannot resolve and the requirement satisfaction relationship references an undefined attribute. The parametric constraint has no value to evaluate, and depending on the tooling, the model instance is either rejected or the constraint is silently skipped. In both cases the engineer receives no diagnostic information about what is missing or how close the design is to satisfying the requirement.

In the proposed SHACL-based approach, the same missing parameter produces a specific readiness violation instead of resulting in structural failure, and the remaining shapes continue to evaluate normally. The RDF graph remains valid because the absence of a triple is then the normal state. Section 7 demonstrates this behavior in which the Sample Ship 1 model satisfies 50 requirements and leaves 151 in pending status, and the validation report identifies exactly which parameters are present, which are missing, and which values violate their thresholds.

The differences between SysML v2 and SHACL do not prevent interoperability, and there are several well-established methodologies for bridging SysML v2 models with semantic web tools. Model Transformation approaches, such as those proposed by Ashari et al. (2021) and Graves (2009), demonstrate how SysML structural diagrams can be systematically converted into OWL ontologies to enable logic-based verification that the native SysML metamodel cannot support. Alternatively, Linked Data standards like Open Services for Lifecycle Collaboration (OSLC) allow SysML blocks to reference external RDF compliance

shapes directly, preserving the system model as the architectural source of truth while offloading regulatory validation to the semantic layer. In this way, the system model can have linked data for validation, and this preserves the use of MBSE and its benefits in traceability and integration. Recent work by Harder et al. (2025) illustrates the potential of this hybrid approach, where SysML's visualization strengths are combined with OWL's capabilities to detect inconsistencies that remain invisible to standard parametric solvers.

4. Ice class rules running example case

The Finnish-Swedish Ice Class Rules serve as a running example throughout the paper, with each stage of the pipeline in Fig. 1 illustrated against the same regulation. We illustrate our proposed method with one continuous model to show the process of extracting, classifying, encoding, and verifying the regulatory rules. To do this, we employ the Finnish Transport and Communications Agency (TRAFICOM), which publishes detailed rules on the ice classes of ships and the provision of icebreaker assistance (Finnish Transport and Communications Agency (Traficom), 2021). These rules establish minimum scantlings and power requirements that a vessel must satisfy to be assigned a given ice class. These rules cover hull structural dimensions such as plate thickness and frame spacing, minimum installed engine power, and propeller strength. Therefore, the rules therefore function as a safety baseline and design constraints where any vessel that falls below these minimums cannot receive the corresponding ice class classification and is not permitted to operate in the associated ice conditions. For Finnish maritime companies that design and operate ships in northern waters, including icebreakers and cargo vessels, these regulations represent a central and recurring design constraint that influences structural arrangements, machinery selection and powertrain sizing from the earliest design stages.

The TRAFICOM ice class regulation contains a mixture of requirement types. In this work, requirements were manually extracted into a structured list and grouped into five categories. Static requirements are basic properties that can be verified directly from design data, such as minimum installed engine power or simple geometric dimensions. Static Calculation requirements involve basic relational or arithmetic operations across several parameters, for example, computing load distribution factors from known dimensions. Dynamic Requirements depend on operating conditions and can only be verified using time-dependent simulations, such as checking propulsion performance or loads under specified ice scenarios. Complex Requirements are not necessarily time-dependent, but involve mathematical expressions or numerical procedures that are too intricate to encode directly in SHACL; in these cases, SHACL is used only to ensure that all variables needed by an external calculation or simulation are present in the model. Finally, Physical Test requirements can only be satisfied through inspection or testing on the built ship and cannot be replaced by simulation; for these, the semantic model can at most record that a physical validation step is required. These five categories partition the regulation so that every clause maps to a specific verification method, whether that is a direct constraint check on the semantic model, a simulation-readiness assessment, or an explicit record that physical inspection is required (Dimyadi et al., 2014).

From the TRAFICOM ice class regulation we extracted 201 individual requirements and classified them into the five categories described: 68 Static, 61 Static Calculation, 38 Physical Test, 22 Dynamic Requirements and 12 Complex Requirements (Fig. 2). The distribution across these categories determines which validation strategy applies to each requirement, direct constraint checking, simulation readiness verification, or physical test tracking. The categories and validation strategy are shown in Fig. 3.

In this paper, the ice class regulation is used as a running example to illustrate both the problem and the proposed solution. First, we identify a set of representative requirements from the regulation that cover different levels of complexity, from simple thresholds to multi-parameter formulas and simulation-dependent conditions. These requirements are then expressed in machine-readable form on top of the ReqTLo system model. Second, we define the corresponding SHACL and SPARQL rules that encode parts of the regulation as explicit validation constraints, and they deterministically check whether the ReqTLo model contains all variables and structures needed to support simulation-based verification of the dynamic requirements. In this way, the ice ship example demonstrates how regulatory text can be transformed into an ontology-based rule set that both prepares a design for simulation and provides an automated and repeatable basis for regulatory compliance checking.

5. Approach description

The proposed approach links maritime regulations directly to a semantic model of the design of the vessel, as illustrated in Fig. 1. It consists of three main components.

1. The system design representation, which uses the ReqTLo ontology to describe the vessel, its components, and its design variables.
2. The regulatory requirement representation, which formalizes extracted clauses as SHACL and SPARQL rules.
3. The validation engine, which automatically checks the ReqTLo model against the regulatory rules and reports compliance status or missing information.

5.1. ReqTLo-based system representation

The ReqTLo ontology described in Section 2.2 serves as the base system model on which the regulatory layer operates. In the ice ship case, a vessel is modeled as an `ssp:System` instance with components such as an engine, propeller, and hull, each linked to observable properties that represent design parameters. For example, a simplified ReqTLo representation of an IA Super ice-class ship and its engine power is:

```
ssp:iceShip a ssp:System ;
    rdfs:label "Ice class IA Super Ship" ;
    ssp:iceClass "IA Super" ;
    ssp:hasComponent ssp:Engine001 .

ssp:Engine001 a ssp:Component ;
    rdfs:label "Main Engine" ;
    ssp:hasVariable ssp:EngineMinPower ;
    ssn:isPropertyOf ssp:iceShip .

ssp:EngineMinPower a sosa:ObservableProperty ;
    rdfs:label "Engine Power" ;
    ssp:hasVariableName "P_eng" ;
    ssp:hasVariableValue 3000.0 ;
    qudt:unit unit:KiloW ;
    ssn:isPropertyOf ssp:Engine001 .
```

Listing 2: ReqTLo representation of engine power variables

This ReqTLo model is the semantic description on which the SHACL rules operate in subsequent steps. The listing uses Turtle syntax, which is a compact way to write RDF triples. Each statement follows a subject–predicate–object pattern. For example, `ssp:iceShip ssp:hasComponent ssp:Engine001` states that the ship contains an engine, and `ssp:EngineMinPower ssp:hasVariableValue 3000.0` states that the engine power is 3000 kW. The semicolon allows multiple predicates to share the same subject, so the block under `ssp:Engine001` describes several properties of the same engine. In this way, the entire ship and its design parameters are represented as a connected graph of typed objects and their relationships.

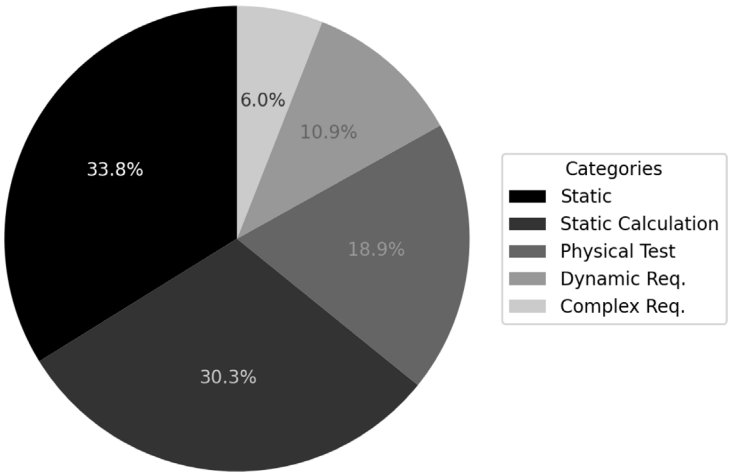

Fig. 2. Breakdown of the different types of regulatory rules present in TRAFICOM.

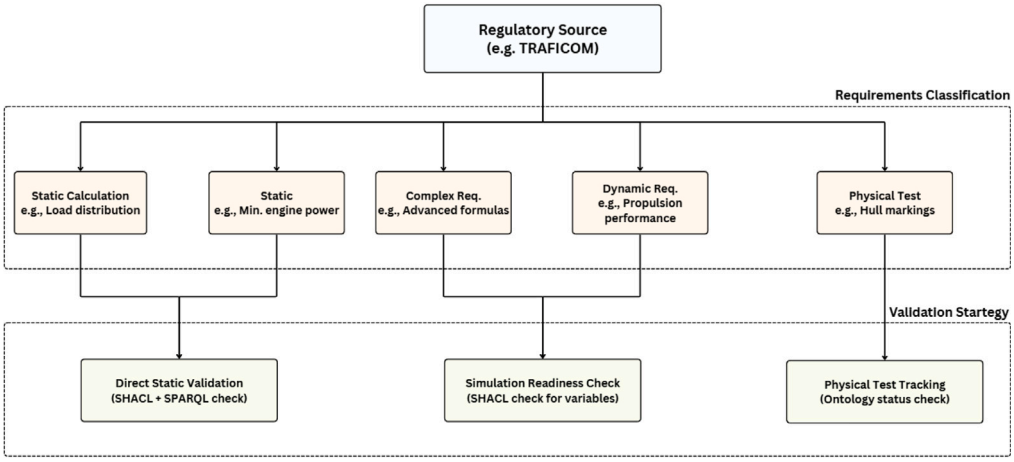

Fig. 3. Overview of the proposed workflow. Regulatory text is analyzed and individual requirements are extracted and classified into five categories.

5.2. Requirement extraction and classification

The TRAFICOM ice class regulation (Finnish Transport and Communications Agency (Traficom), 2021) is used as a regulatory source. All individual requirements were manually extracted into a structured spreadsheet and classified into five categories: Static, Static Calculation, Dynamic Requirements, Complex Requirements and Physical Test (see Section 4). This classification distinguishes between requirements that can be checked directly on design parameters, those that require simple calculations, those that depend on time-varying behavior, those that need more advanced numerical methods, and those that necessarily remain physical.

This step clarifies which parts of the regulation can be encoded as direct SHACL constraints on the ReqTLo model, which parts are handled through simulation readiness checks, and which parts must be verified by physical tests.

5.3. Mapping regulations to ReqTLo

For each extracted requirement, the relevant system entities and variables are identified and mapped to ReqTLo concepts. For example, the rule “ships of ice class IA Super must have a minimum engine power of 2800 kW” is linked to the ReqTLo ship instance `ssp:iceShip`, its ice class property, and the associated engine power variable.

Where existing ReqTLo and reused ontologies are not sufficient, additional classes and properties are introduced to represent maritime-specific aspects such as ice load parameters on hull structures. After this step, every requirement refers to well-defined classes and properties in the ontology, which is necessary for writing SHACL and SPARQL rules in a consistent way.

5.4. Encoding SHACL and SPARQL rules

The regulations are then encoded as SHACL shapes and SPARQL query constraints operating on the ReqTLo model. The five requirement categories are treated differently. Static requirements are encoded as SHACL shapes that check simple value constraints. For the minimum engine power rule, a shape targets the ship instance and verifies that the engine power is above the threshold for the given ice class:

```
ice:MinimumEnginePowerShape a sh:NodeShape ;
    sh:targetNode ssp:iceShip ;
    sh:message "Ship does not meet minimum engine
     power." ;
    sh:sparql [
        sh:message "Engine power ({?actualPower}
     kW) is
        below the minimum for {?iceClass} ice
     class." ;
        sh:select """
        PREFIX ssp:
     <https://w3id.org/mtl-requirements/ssp#>
        PREFIX sosa: <http://www.w3.org/ns/sosa/>
        PREFIX ssn: <http://www.w3.org/ns/ssn/>
```

```
      SELECT \stmdollarthis ?actualPower
    ?iceClass WHERE {
          \stmdollarthis ssp:iceClass ?iceClass .
          \stmdollarthis ssp:hasComponent
    ?engine .
          ?engine a ssp:Component .
          ?engine ssp:hasVariable ?powerVar .
          ?powerVar a sosa:ObservableProperty .
          ?powerVar ssp:hasVariableName "P_eng" .
          ?powerVar ssp:hasVariableValue
    ?actualPower .

          FILTER(?iceClass = "IA Super" &&
    ?actualPower < 2800.0)
       }
       """
  ] .
```

Listing 3: SHACL shape to extract engine power from the ontology representation

This encodes the requirement that if the installed power is below 2800 kW for an IA Super ship, the validation engine reports a violation with the actual value.

Static Calculation requirements use SHACL with SPARQL arithmetic to compute derived values from several parameters and compare them to regulatory formulas. For example, the load distribution factor f_4 is calculated as:

$$f_4 = 1 - 0.2 \cdot \frac{h}{s} \tag{1}$$

where h is the load area height and s is the frame spacing. A SHACL rule retrieves h and s from the ReqTLo model, computes f_4 and checks that the stored value matches within a tolerance.

Complex requirements involve formulas that exceed SPARQL's native capabilities, such as expressions with radicals or more advanced numerical methods. For these, the SHACL rules do not attempt to re-implement the full calculation. Instead, they check that the ReqTLo model contains all the parameters required by the external simulation or analysis tool. In other words, SHACL is used to verify simulation readiness for these requirements.

An example of a Complex requirement is the determination of ice resistance in a brash ice channel (R_{CH}) as defined in the TRAFICOM rules. As illustrated in Eq. (2), this calculation is based on variable geometric parameters and fractional exponents that cannot be evaluated using standard SPARQL arithmetic. Consequently, the model does not attempt to compute R_{CH} directly within the semantic layer. Instead, the approach enforces a simulation readiness check where the ontology formally defines `ssp:IceResistance` as a required observable property of the hull (Listing 4), and the corresponding SHACL rule validates its existence in the system graph (Listing 5). This verification step guarantees that the model is structurally prepared to receive and store the resistance values generated by the external physics solver.

$$R_{CH} = C_1 + C_2 + C_3 C_\mu (H_F + H_M)^2 (B + C_\psi H_F) + C_4 L_{PAR} H_F^2 + C_5 \left(\frac{LT}{B^2}\right)^3 \frac{A_{wf}}{L} \tag{2}$$

```
ssp:IceResistance a sosa:ObservableProperty ;
    rdfs:label "Ice resistance in Newton of the ship
     in a channel with brash ice." ;
    ssp:hasVariableName "R_ch" ;
    ssn:isPropertyOf ssp:Hull001 .
```

Listing 4: Ontology definition for Ice Resistance

```
ice:IceResistanceReadinessShape a sh:NodeShape ;
    sh:targetNode ssp:Hull001 ;
    sh:message "Simulation Readiness Check: Ice
     Resistance (R_ch) definition is missing." ;
    sh:sparql [
        sh:message "The Hull is missing the
     'ssp:IceResistance' property required for
     simulation output." ;
        sh:select """
            PREFIX ssp:
     <https://w3id.org/mtl-requirements/ssp#>
            PREFIX ssn: <http://www.w3.org/ns/ssn/>
            SELECT \stmdollarthis
            WHERE {
                FILTER NOT EXISTS {
     ssp:IceResistance ssn:isPropertyOf
     \stmdollarthis . }
            }
         """
    ] .
```

Listing 5: SHACL Simulation Readiness Shape

To verify the handling of Complex requirements, a second validation case was performed targeting the simulation readiness of the hull. The `ssp:IceResistance` property was intentionally omitted from the system model to simulate a design state that is structurally incomplete for physics-based analysis. The validation engine successfully detected this omission, flagging a specific readiness violation (Listing 6) rather than a simple parameter threshold failure. This confirms that the framework can distinguish between design non-compliance (e.g., insufficient power) and model incompleteness (missing simulation interfaces).

```
Conforms: False
Results (1):
Constraint Violation in SPARQLConstraintComponent:
  Severity: sh:Violation
  Source Shape: ice:IceResistanceReadinessShape
  Focus Node: ssp:Hull001
  Message: The Hull is missing the
     'ssp:IceResistance' property required to
     store the output of the complex ice resistance
     calculation.
  Message: Simulation Readiness Check: Ice
     Resistance (R_ch) definition is missing.
```

Listing 6: Validation report for Ice Resistance readiness check

Dynamic Requirements depend on varying behavior and are evaluated through simulation. Here, SHACL rules ensure that all necessary time-dependent variables are present and correctly attached to the relevant components in the ReqTLo model. For example, the TRAFICOM rules define the maximum backward force on the propeller as

$$F_b = 27(nD)^{0.7} \left(\frac{EAR}{Z}\right)^{0.3} D^2, \tag{3}$$

where n is the rotational speed, D is the diameter of the propeller, EAR is the expanded area ratio, and Z is the number of blades. In the ontology, these variables are modeled as observable properties of the propeller:

```
ssp:MainPropeller a ssp:Component ;
    ssp:hasVariable ssp:BackwardShipForce,
     ssp:ForwardShipForce ;
    ssn:isPropertyOf ssp:iceShip .

ssp:BackwardShipForce a sosa:ObservableProperty ;
    ssp:hasVariableName "F_b" ;
    ssn:isPropertyOf ssp:MainPropeller .
```

Listing 7: Variable representation when explicit calculation is not possible with SHACL

The corresponding SHACL rule then checks that both forward and backward forces are defined for the propeller, so that the simulation can compute F_b and related quantities. The actual numerical verification of the requirement is left to the simulation tool.

Physical Test requirements cannot be replaced by simulation. In the ontology, each such clause is represented as an individual of the class `req:PhysicalTestRequirement` linked to the relevant ship and annotated with a verification status. For example, the requirement: "Ship's sides must be provided with a warning triangle and draught mark." is modeled as `req:WarningTriangleAndDraughtMarkReq` with a property `req:hasVerificationStatus` whose value is one of `"pending"`, `"passed"` or `"failed"`:

```
req:WarningTriangleAndDraughtMarkReq a
    req:PhysicalTestRequirement ;
   rdfs:label "Ship's sides must be provided with a
    warning triangle and draught mark." ;
   req:appliesTo ssp:iceShip ;
   req:hasVerificationStatus "pending" .
```

Listing 8: Verification status is added to each requirement for traceability

A SHACL shape `req:PhysicalTestStatusShape` then targets all `req:PhysicalTestRequirement` instances and checks that this status property is present and has one of the allowed values:

```
req:PhysicalTestStatusShape a sh:NodeShape ;
   sh:targetClass req:PhysicalTestRequirement ;
   sh:message "Physical test requirement is
    missing a valid verification status." ;
   sh:property [
      sh:path req:hasVerificationStatus ;
      sh:minCount 1 ;
      sh:datatype xsd:string ;
      sh:in ("pending" "passed" "failed") ;
      sh:message "Physical test must be marked as
    'pending', 'passed' or 'failed'."
   ] .
```

Listing 9: SHACL constraints on the verdict for physical tests

The shape ensures that every Physical Test requirement has a recorded verification status, but the shape does not prescribe how that status is entered into the model. The status update is represented as a replacement for one RDF triple in the relevant requirement instance. For example, after a quality engineer has inspected the hull markings on the constructed vessel, the status of `req:WarningTriangleAndDraughtMarkReq` can be changed from `req:PendingStatus` to `req:PassedStatus`. The SPARQL 1.1 Update specification (Gearon et al., 2013) provides the `DELETE/INSERT` operation needed for this update.

The same update pattern can be implemented in several deployment settings. A simple file-based workflow can update the RDF graph directly. A triplestore-backed workflow can submit the update through the SPARQL 1.1 Graph Store HTTP Protocol (Ogbuji, 2013), which provides a standard interface for reading and writing RDF graphs. An organization that manages verification evidence through a Product Lifecycle Management (PLM) or Quality Management System (QMS) can route the same status change through an Open Services for Lifecycle Collaboration (OSLC) interface (Amsden and Berezovskyi, 2021). These alternatives differ in integration architecture, but all produce the same semantic result, namely an updated verification status triple that can be checked again by SHACL. Regulatory amendments require a different form of maintenance, where the SHACL shapes graph is revised instead of the requirement status data, as discussed in Section 8.

5.5. Validation process and simulation readiness

Once a candidate ship design has been modeled in ReqTLo, all SHACL and SPARQL rules derived from the ice class regulation are evaluated using the pySHACL engine (Ke et al., 2024). The engine applies the shapes to the ReqTLo RDF graph and produces a validation report.

For Static and Static Calculation requirements, the report lists satisfied and violated constraints together with messages that identify the offending values. For Dynamic, Complex and Physical Test requirements, the report focuses on the presence or absence of required variables, links to components and placeholders for physical test evidence. Any missing elements are reported as validation failures in that the semantic model does not contain the necessary information to carry out the verification process against the regulations.

The validation therefore serves two complementary functions. It acts as a direct compliance check for requirements whose logic can be expressed in SHACL, and it acts as a simulation-readiness assessment for requirements that must be evaluated by an external tool. Simulation readiness, in this context, means that the ReqTLo model contains all variables, component relationships, and unit annotations that a given computational method requires as input, regardless of whether that method is a time-domain simulation, a static numerical solver, or an iterative calculation procedure. A model that passes the readiness shapes is structurally complete for external verification, and the specific choice of solver is left to the implementation.

For Dynamic and Complex requirements, the framework confirms the existence of all necessary variables to carry out the verification. This means that external tools are required to carry out compliance with these requirements. Therefore, a pass verdict in the proposed semantic model in the case of dynamic and complex requirements indicates simulation-readiness instead of compliance.

To illustrate how violations are reported, we modify the engine power in the ReqTLo model to a non-compliant value. In Listing 10 the engine power is reduced from `3000.0` kW to `1000.0` kW, which is below the minimum required for an IA Super ice-class ship:

```
ssp:EngineMinPower a sosa:ObservableProperty ;
   rdfs:label "Engine Power" ;
   ssp:hasVariableName "P_eng" ;
   ssp:hasVariableValue 1000.0 ;
   qudt:unit unit:KiloW ;
   ssn:isPropertyOf ssp:Engine001 .
```

Listing 10: Non-compliant engine power definition in ReqTLo model

Running the SHACL validation (using `pySHACL`) on this modified model produces the following extract from the validation report:

```
Conforms: False
Results (1):
Constraint Violation in SPARQLConstraintComponent:
   Severity: sh:Violation
   Source Shape: ice:MinimumEnginePowerShape
   Focus Node: ssp:iceShip
   Message: Engine power (1000.0 kW) is below the
    minimum 2800 kW required for IA Super ice
    class.
   Message: Ship does not meet TRAFICOM minimum
    engine power requirements.
```

Listing 11: Excerpt from SHACL validation report for non-compliant engine power

The validation engine identifies the ship instance `ssp:iceShip` as the focus node and reports that the engine power of `1000.0` kW violates the minimum threshold of `2800.0` kW for the IA Super ice class. This example shows how SHACL validation provides concrete design-level feedback when a static regulatory requirement is not satisfied.

Beyond this specific example, the method shows how individual clauses of a maritime regulation can be turned into executable checks that operate directly on a ship design model. Each requirement is linked to a concrete object in the design, such as the ship instance `ssp:iceShip` and its engine parameters, and violations are reported in terms of those objects and their numerical values. For a naval architect or systems engineer, this means that feedback on non-compliance is obtained at the same level as other design decisions, rather than through a separate manual review of textual rules.

6. Model scale and computational performance

In this section, we report on the scale of the semantic model needed to capture the regulatory rules in the TRAFICOM document using the method proposed in this paper. These metrics address two questions that are relevant for adoption in engineering practice. The first is whether the ontology and its associated shapes remain manageable on the scale of a complete maritime regulation. The second is whether automated validation completes fast enough to support iterative design workflows where engineers modify parameters and re-check compliance frequently.

6.1. Model and shape graph metrics

The complete ReqTLo-based ship model modeled in this paper contains 1 921 RDF triples, and the SHACL shapes graph contains 736 RDF triples. The data graph represents 4 system components (`ssp:Component` instances) and 131 observable properties (`sosa:ObservableProperty` instances) that capture hull geometry, propulsion parameters, ice load variables, and structural dimensions. Together, these triples encode a complete IA Super ice-class ship model with all relevant variables referenced by the TRAFICOM regulation.

The 201 extracted regulatory requirements are covered by 105 SHACL node shapes that target 21 distinct focus nodes in the data graph. The shape count is lower than the requirement count because several regulatory clauses share the same verification logic and can be evaluated by a single shape. For example, a shape that checks whether all required hull geometry variables satisfy the readiness condition for multiple Complex requirements that reference those same variables. Conversely, a small number of shapes contain multiple embedded SPARQL queries that each address a different aspect of the same regulatory clause. The shapes graph contains 119 distinct SPARQL queries in total.

The overwhelming majority of shapes (101 out of 105) use SHACL-SPARQL constraints, and only 4 shapes rely exclusively on SHACL-Core property constraints. This distribution reflects the nature of the regulatory domain. Ice class requirements predominantly involve arithmetic comparisons, lookup-table verification, multi-parameter calculations, and structural completeness checks, all of which require the expressiveness of embedded SPARQL queries. The few SHACL-Core shapes handle simple cases such as verifying that a verification status property is present and has one of the permitted string values.

The full validation of the data graph against all 105 SHACL shapes was executed 50 times on a workstation equipped with an AMD Ryzen 7 7840HS processor (3.80 GHz, 8 cores) and 16 GB of RAM, using pySHACL running on Python 3.8.10 inside Ubuntu 24.04.3 via WSL. The mean wall-clock time for all 50 runs was **2.54 seconds**. Table 2 summarizes the model metrics and execution time.

Table 2
Model scale and validation performance metrics.

Metric	Value
Data graph triples	1 921
Shapes graph triples	736
SHACL node shapes	105
of which SHACL-SPARQL	101
of which SHACL-Core only	4
Distinct SPARQL queries	119
Focus nodes targeted	21
`sosa:ObservableProperty` instances	131
`ssp:Component` instances	4
Regulatory requirements covered	201
Mean validation time (50 runs)	2.54 s
Hardware	AMD Ryzen 7 7840HS, 16 GB

The 2.54 s execution time includes parsing both Turtle files, constructing the in-memory RDF graphs, evaluating all 119 SPARQL queries against the data graph, and serializing the validation report. There are more distinct SPARQL queries than SHACL node shapes since each node shape can contain multiple different SPARQL queries to validate a given requirement.

This computational profile supports a workflow in which SHACL validation is structured as a continuous background check throughout the design process. Manual cross-checking of 201 requirements against a model with 131 variables would require an engineer to locate each parameter, verify its value or confirm its presence, cross-reference lookup tables for the appropriate ice class, and record the outcome. The literature on automated compliance check in adjacent engineering domains consistently characterizes this manual process as labor-intensive, error-prone, and difficult to repeat consistently, given the complexity of regulations (Nuyts et al., 2023; Zhang and El-Gohary, 2017). To estimate the manual effort, consider that for each of the 201 requirements an engineer must read the regulatory clause, locate the relevant parameters in the design, cross-reference the applicable lookup table for the declared ice class, perform any required calculation, and record the result. At a conservative average of 10 to 20 min per requirement, the total effort is on the order of 30 to 70 engineering hours. The automated SHACL validation completes the same check in 2.54 s and can be re-executed after every design change at negligible cost.

The automated approach eliminates the possibility of overlooking a requirement, produces identical results for identical inputs regardless of who runs it, and provides specific diagnostic messages that identify the exact component, variable, and numerical value responsible for each violation. Furthermore, the distinction between requirements that are not yet verifiable (pending due to missing model data) and requirements that are non-compliant (failed due to incorrect values) is maintained automatically through the verification status mechanism, and this distinction is difficult to track reliably in spreadsheet-based workflows as designs evolve over many iterations.

7. Verification against sample ship data

The preceding sections establish the ontological framework and define SHACL shapes for 201 extracted requirements. In this section, we demonstrate how these shapes operate against concrete ship design data with independently verifiable outcomes. This section instantiates the semantic model with sample vessel data from the TRAFICOM regulation's verification annex and runs the full validation cycle, the ship parameters are shown in this report (Finnish Transport Safety Agency (Trafi), 2010) and listed in Table 3. The objective is to show how the status of the requirements transitions from pending to passed or failed as design information becomes available, and how the aggregate compliance picture changes as a result.

Table 3
Sample Ship 1 parameters from TRAFICOM Annex I, Table 2.

Parameter	Symbol	Value
Ice class	–	IA Super
Waterline angle at $B/4$	α	24°
Stem rake (bulbous bow)	φ_1	90°
Bow rake at $B/4$	φ_2	30°
Length between perpendiculars	L	150 m
Maximum breadth	B	25 m
Ice class draught	T	9 m
Bow length	L_{BOW}	45 m
Parallel midbody length	L_{PAR}	70 m
Bow waterline area	A_{wf}	500 m^2
Propeller diameter	D_P	5 m
Propeller configuration	–	1/CP
Published minimum power	P_{min}	7840 kW

7.1. Sample ship data

The TRAFICOM ice class regulation includes a verification annex (Annex I) that provides the hull geometry and propulsion parameters of nine sample ships in addition to independently calculated minimum engine power values (Finnish Transport Safety Agency (Trafi), 2010). These sample ships are intended for engineers to validate their own powering calculations against published reference answers. Therefore, they serve as an ideal external benchmark for the semantic compliance framework because the expected outcomes are known in advance and can be checked without access to proprietary design data. Sample Ship 1 is selected as the primary test case because it represents the ice class IA Super, the most demanding category, and it performs the full calculation of ice resistance, including the terms of the consolidated surface layer C_1 and C_2 that apply only to IA Super vessels. Table 3 lists the input parameters for this vessel.

7.2. Verification process

With the sample ship parameters in Table 3 instantiated in the ReqTLo model, the validation engine can directly evaluate requirements from three of the five categories: Static, Static Calculation, and Complex. Static requirements are evaluated by comparing declared values with regulatory thresholds or lookup tables. For Sample Ship 1, the verification confirms that the declared ice class "IA Super" belongs to the recognized set, that the engine power of 7840 kW exceeds the 2800 kW minimum, that the ice load height $h = 0.35$ m matches the prescribed value, that the brash ice thickness $H_M = 1.0$ m equals the IA Super tabulated value, and that the propeller diameter $D_P = 5$ m is declared as an observable property of the propeller component. Each verification is performed by a dedicated SHACL shape that targets the relevant node in the ReqTLo graph and reports a violation if the condition fails.

Static Calculation requirements extend this process to derived quantities that combine multiple parameters. The coefficient K_e coefficient for a single controllable-pitch propeller should be equal to 2.03 according to the TRAFICOM lookup table in Section 3.2.2 (Finnish Transport and Communications Agency (Traficom), 2021), and the SHACL shape verifies this by retrieving the count of the propeller, the type of propeller and the stored value K_e from the model and comparing the stored value with the expected value. The friction coefficient C_μ should reach the minimum of 0.45 after calculating the bow angles φ_2 and α, and the clamped ratio $(LT/B^2)^3 \approx 10.08$ should fall within the permitted range of 5 to 20. These shapes use SPARQL arithmetic within their `sh:sparql` constraints to compute the expected values and flag discrepancies.

Complex requirements follow a different verification path because they involve mathematical expressions that exceed SPARQL's native arithmetic capabilities. The SHACL shapes for these requirements verify the simulation readiness by validating that all required input variables are present and correctly linked in the ReqTLo model. For example, the SHACL rule on the readiness of the hull geometry evaluates that the eight parameters referenced by the formula R_{CH} formula (L, B, L_{BOW}, L_{PAR}, A_{wf}, α, φ_1, φ_2) are defined as `sosa:ObservableProperty` instances attached to their respective components. The P_{min} formula readiness shape similarly confirms that R_{CH}, K_e, and D_P are all available. The actual ice resistance calculation is then performed by an external solver, which computes $P_{\mathrm{min}} = 7840$ kW for Sample Ship 1. This result matches the published TRAFICOM reference value and confirms both the correctness of the external computation and the completeness of the semantic model that supplied its inputs.

```
# Before verification
req:MinimumEnginePowerReq a
     req:StaticRequirement ;
    rdfs:label "Engine output must not be less than
        2800 kW for IA Super." ;
    req:appliesTo ssp:iceShip ;
    req:validatedBy ice:MinimumEnginePowerShape ;
    req:hasVerificationStatus req:PendingStatus .

# After verification (engine power 7840 kW >= 2800
     kW)
req:MinimumEnginePowerReq a
     req:StaticRequirement ;
    rdfs:label "Engine output must not be less than
        2800 kW for IA Super." ;
    req:appliesTo ssp:iceShip ;
    req:validatedBy ice:MinimumEnginePowerShape ;
    req:hasVerificationStatus req:PassedStatus .
```

Listing 12: Requirement status before and after verification.

Table 4 shows a representative subset of the requirements verified in this process. The full verification covers 50 of the 201 formalized requirements, spanning Static, Static Calculation and Complex categories. The remaining requirements correspond to hull structural calculations that require frame spacing, plate thickness, and material properties not provided in Annex I, propulsion machinery requirements that depend on blade geometry and torsional vibration analysis, and Physical Test requirements that can only be resolved by inspection of the constructed vessel. Despite this, the method proposed in this paper ensures traceability and continuous verification of design status and conformance with the regulatory rules.

```
# Before verification of Sample Ship #1
Conforms: False
Results (1):
  Source Shape: ice:AllRequirementsVerifiedShape
  Focus Node: ssp:iceShip
  Message: 201 requirements still have pending
           verification status.

# After verification of Sample Ship #1
Conforms: False
Results (1):
       Severity: sh:Violation
       Source Shape:
     ice:AllRequirementsVerifiedShape
       Focus Node: ssp:iceShip
       Message: 151 requirements still have
     pending verification status.
       Message: Not all requirements have been
     verified.
```

Listing 13: Aggregate validation result before and after verification.

Table 4
Representative requirements verified against Sample Ship #1. Note that for the complex requirements case, a pass here indicates the existence of all variables needed to complete an analysis to affirm compliance. Therefore, pass here for the complex category is for all the variables needed for simulation being present.

Requirement	Category	Verification	Verdict
Ice class validity (§1.1)	Static	"IA Super" belongs to the recognized set {IA Super, IA, IB, IC, II, III}	Pass
Minimum engine power (§3.2)	Static	$P = 7\,840 \geq 2\,800$ kW threshold for IA Super	Pass
Brash ice thickness H_M (§3.2.2)	Static	$H_M = 1.0$ m matches IA Super prescribed value	Pass
K_e value consistency (§3.2.2)	Static Calc.	Single CP propeller: $K_e = 2.03$ per lookup table	Pass
C_μ minimum (§3.2.2)	Static Calc.	$C_\mu \geq 0.45$ computed from $\varphi_2 = 30°$, $\alpha = 24°$	Pass
$(LT/B^2)^3$ range (§3.2.2)	Static Calc.	≈ 10.08, within permitted range $[5, 20]$	Pass
Hull geometry readiness (§3.2.1)	Complex	All eight R_{CH} input variables present in model	Pass
$P_{\min}$ formula readiness (§3.2.2)	Complex	R_{CH}, K_e, D_P defined; external result = 7 840 kW	Pass

7.3. Status update and closed-loop feedback

Each verified requirement is recorded in the ontology by updating its `req:hasVerificationStatus` property from `req:Pending Status` to either `req:PassedStatus` or `req:FailedStatus`. Listing 12 illustrates this transition for the minimum engine power requirement. The only change between the pre-verification and post-verification states of the requirement instance is the object of the `req:hasVerificationStatus` triple, and this single-triple update is the mechanism through which external verification results are fed back into the semantic model. The same pattern applies to all requirement categories. Static requirements are resolved by a SHACL shape evaluating them against the model data, Static Calculation requirements by the shape computing a derived value and confirming or rejecting it, Complex requirements by a passed readiness check and a result from an external solver, and Physical Test requirements by an inspector recording the outcome in the model.

The aggregate effect of these transitions is visible through the `AllRequirementsVerifiedShape` introduced in Section 5. This shape targets the ship instance and counts all requirements whose status remains `req:PendingStatus`. Listing 13 shows the validation output before and after the verification of the data from the Sample Ship 3. The pending count drops from 201 to 151, and the remaining pending requirements are directly attributable to design information that is not yet available in the model.

8. Discussion

This work shows that maritime regulatory requirements can be classified by verification type and encoded as machine-executable constraints over a semantic ship design model, supporting compliance verification that operates both directly on design parameters and as a readiness evaluation for simulation-based verification. The five-category scheme classified 201 requirements in the TRAFICOM regulation without ambiguity. The Static and Static Calculation requirements account for about two thirds of the total and can be verified directly on the semantic model without simulation, providing immediate feedback and avoiding the computational cost of time-domain simulations. Dynamic and Complex requirements (roughly one sixth) require simulation-readiness verification, while Physical Test requirements (the remaining fifth) are tracked with explicit verification status.

Regulations can have different interpretations depending on the use case and the person reading them. Because of this, the boundaries between categories may not be fixed and depend on the capabilities of the verification tools available to the engineer. A requirement that is currently classified as a Physical Test may shift toward virtual verification as higher-fidelity simulation tools become available, and thereby become a dynamic or complex requirement. The semantic model supports this transition because the requirement instance, its link to the relevant component, and its verification status all remain in the same RDF graph regardless of whether the verification method changes. Therefore, the classification does not lock requirements into a permanent verification path, and the same linked semantic structure can represent regulations beyond ice class rules since the categories reflect general distinctions in how requirements are verified.

The validation process is progressive because each requirement status can be updated as new design evidence becomes available. Additional design parameters from detailed structural engineering allow the hull scantling requirements in Section 5.4 to be evaluated and moved from pending to passed or failed. Completed torsional vibration analyses and fatigue calculations allow the Dynamic requirements in Section 5.5 to be updated in the semantic model in the same way. Physical Test requirements remain pending until inspection or testing on the constructed vessel provides the required evidence. Each update reduces the number of unresolved requirements and narrows the set of outstanding compliance obligations. The practical implication of this is that compliance verification runs continuously alongside the design process, and with each new parameter, analysis, or inspection result updating, the model updates the verification status of the affected requirements. The engineer can therefore re-run the SHACL validation at any point in the design lifecycle and obtain a compliance summary that reflects both verified results and the information that still remains unavailable.

This approach treats regulation modeling, simulation readiness, and requirement status as a unified semantic layer on top of an existing system ontology. The method recognizes that different types of requirements require different verification treatments and uses SHACL accordingly, which the previous methods do not address (Chen et al., 2020; Schmidt et al., 2007; Lee and Gandhi, 2005). The combination of a requirement ontology with an external SHACL constraint layer allows regulatory rules to be attached to existing terminology and model conventions, and this makes compliance checking more consistent across engineering teams without requiring a universal domain vocabulary. This aligns with larger efforts to standardize terminology and meaning in models-based engineering contexts (De Haan et al., 2024). Moreover, the separation of concerns between the descriptive system model and the normative rule set means that regulatory overlays from different authorities can target the same ship components and be enforced together without modification to the underlying system ontology.

The 201 TRAFICOM requirements were extracted and formalized manually. Maritime frameworks such as SOLAS and MARPOL contain thousands of clauses, and the same manual process would not scale to that volume. Scaling the approach means making two tasks less costly, extracting requirements from regulatory text and writing the corresponding SHACL shapes. The five-category classification scheme helps with both. It is a fixed, human-defined template that tells the person writing a shape what kind of constraint each clause needs, whether a direct value check, an arithmetic comparison, a simulation-readiness query or a status-tracking constraint. A SHACL shape also does not have to be written in one specific way to be correct. Different shapes can encode the same clause, and a shape is correct as long as it gives the right pass-or-fail verdict on a known design. A direction for future work is to use Large Language Models (LLMs) for both tasks, extracting clauses from regulatory text and translating

them into SHACL shapes, as explored in prior work on LLM-based rule extraction (Heimsbakk and Torkelsen, 2023). The five-category scheme would guide this step by telling the model which shape pattern each type of clause needs. The LLM would not define or change the categories, which remain human-defined. Because the output needs to conform to a strict machine-checkable representation, the pySHACL processor immediately detects malformed shapes. However, the LLM output is not reliable enough to trust without checking. Generated shapes can be wrong, so for safety-critical rules, such as ice-class requirements, they must be verified before use, with a human reviewing the result.

The status updates described in Section 5.4 concern verification evidence for an existing set of regulatory rules. Lifecycle management of the regulatory knowledge base is a separate practical concern, as maritime regulations change often. The Finnish-Swedish Ice Class Rules, for example, were revised in 2021 and again in 2025. An encoded constraint set is useful while staying aligned with the regulation in force. Therefore, the encoding process should remain manageable. The proposed approach supports this because the regulatory layer is kept separate from the system model. When a regulation is amended, the engineer edits the SHACL shapes graph. The ReqTLo ontology does not change, and since the encoding is declarative, revising a clause means editing a constraint instead of rewriting the program code. The five-category classification helps even further because it tells the engineer in advance what kind of edit a clause needs. This separation of shapes from the system model is the practice recommended by mature SHACL tools. Editors such as Protégé with the SHACL4Protégé plugin Ekaputra and Lin (2016) support it directly. The plugin provides a dedicated interface for editing shapes and validating the model against them. Updating the knowledge base after an amendment is still a manual step, because an engineer has to identify the affected clause and revise its shape. However, this work is still organized, and the engineer does not search through scattered code files one by one, but works with a small set of constraints in interfaces built for this purpose of semantic model management.

The use of SPARQL as the query language for the proposed semantic model limits the numerical logic that can be evaluated directly inside the semantic layer. SPARQL is designed primarily for graph pattern matching and value filtering over RDF data, and its portable arithmetic support is limited to basic numeric operators such as addition, subtraction, multiplication, and division. This is sufficient for many Static Calculation requirements, especially threshold, lookup-table, and simple algebraic verification. However, formulas that require fractional exponents, roots, transcendental functions, or iterative numerical procedures are difficult to express in portable SHACL-SPARQL shapes. Some SHACL and SPARQL processors provide implementation-specific extension functions for such operations, but relying on these functions reduces portability and makes the validation layer dependent on a particular tool. For this reason, the proposed framework treats advanced formulas as Complex requirements when their numerical evaluation would require non-standard functions or procedural algorithms. In these cases, SHACL is used to verify that the semantic model contains the required variables, units and component relationships, and the numerical evaluation is delegated to an external solver or simulation tool.

Future work should investigate how Large Language Models (LLMs) can support two tasks in the proposed framework, namely generating candidate SHACL shapes from regulatory clauses and translating technical validation messages into explanations for engineers. The LLM would not determine compliance directly, since regulatory verdicts in a safety-critical setting must be produced by deterministic and auditable procedures that apply explicit rules to traceable design data. This separation is necessary because LLM outputs can be affected by hallucinations, stochastic variation across runs and limited explainability regarding how a specific classification or rule encoding was produced. The role of the LLM would therefore be limited to proposing candidate rule encodings or explanatory text that must be checked before use. Oracle-guided verification provides one possible control mechanism, since generated shapes can be accepted only when they reproduce the expected pass-or-fail verdicts on benchmark cases with known outcomes (Zhang et al., 2023; Cunha and Macedo, 2025; Faria et al., 2026). The TRAFICOM Annex I sample ships used in Section 7 provide such a reference set for ice-class calculations, and similar annexes could support testing for other maritime regulations. This workflow still requires expert review, since errors in clause extraction, category assignment or generated explanations could create misleading compliance evidence. Future studies should therefore measure the accuracy of LLM-assisted rule generation, the effort required for human correction and the reliability of generated explanations across larger and more diverse regulatory rule sets (Leinonen et al., 2023; Balse et al., 2023; Hassani, 2024).

Beyond rule extraction, future work should evaluate the approach in industrial ship design projects to quantify its impact on engineering effort and development timelines. In particular, studies can measure whether earlier SHACL-based checks shift simulations to earlier stages, reduce late rework caused by missing parameters or violated thresholds, and clarify when simulation evidence is sufficient versus when physical testing remains necessary. This would provide concrete evidence of how the method affects the timing and balance of verification activities in practice.

9. Conclusion

This paper proposed a method for classifying maritime regulatory requirements by verification type and encoding them as machine-executable SHACL constraints on a semantic ship design model. The five-category classification (Static, Static Calculation, Dynamic, Complex and Physical Test) was applied to 201 requirements of the TRAFICOM Finnish-Swedish Ice Class Rules, and verification against independently published sample ship data resolved 50 requirements to passed status while the remaining 151 were traced to specific design parameters not yet available in the model. The results demonstrate that regulatory clauses can function as active, traceable constraints on an evolving design model and that the semantic structure maintains a clear distinction between requirements that are non-compliant and requirements that are not yet verifiable. This distinction supports iterative design workflows by giving engineers continuous feedback on compliance status without requiring a complete model, and reduces the risk that missing variables or violated thresholds are discovered only after costly simulations or sea trials. The approach is not limited to ice class rules and can be extended to other maritime regulations such as emission limits or stability criteria by adding independent SHACL shapes graphs that target the same underlying system model. Future work will focus on semi-automating the extraction of regulatory clauses and their translation into SHACL shapes, and on evaluating the method in industrial ship design projects to quantify its effect on engineering effort and the timing of verification activities.

CRediT authorship contribution statement

Haitham Al-Shami: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Conceptualization. **Riku Ala-Laurinaho:** Writing – review & editing, Supervision, Resources, Methodology, Funding acquisition, Conceptualization. **Raine Viitala:** Writing – review & editing, Supervision, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

Business Finland funded this research under Grant 243/31/2022 'Co-Des-Digital transformation of collaborative powertrain design'.

References

Al-Shami, H., Ala-Laurinaho, R., Eklund, M., et al., 2025. ReqTLo: Requirements temporal logic ontology for simulation-based validation of systems in digital twin enabled collaborative design. URL https://www.techrxiv.org/. Preprint. TechRxiv.

Ala-Laurinaho, R., Autiosalo, J., Laine, S., Hakonen, U., Viitala, R., 2024. Paradigm shift in mechanical system design: toward automated and collaborative design with digital twin web. Softw. Syst. Model. 1–20.

Alblas, G., Pruijn, J., 2024. Are current shipbuilding cost estimation methods ready for a sustainable future? A literature review of cost estimation methods and challenges. Int. Shipbuild. Prog. 71 (1), 3–28. http://dx.doi.org/10.3233/ISP-230009.

Amsden, J., Berezovskyi, A., 2021. OSLC core version 3.0. OASIS Open. URL https://www.oasis-open.org/standard/oslc-core-version-3-0/. Open Services for Lifecycle Collaboration (OSLC) Open Project.

Antoniou, G., Harmelen, F.v., 2009. Web ontology language: OWL. In: Staab, S., Studer, R. (Eds.), Handbook on Ontologies. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 91–110. http://dx.doi.org/10.1007/978-3-540-92673-3_4.

Ashari, A., Sari, A., Wardhana, H., 2021. An extended rule of the SysML requirement diagram transformation into OWL ontologies. Int. J. Intell. Eng. Syst. 14, 506–515. http://dx.doi.org/10.22266/ijies2021.0228.47.

Astrup, O., Aae, O., Kuś, T., Uyanik, O., Biterling, B., Bars, T., Polini, M., G, V., Yu, K., Seppälä, T., Son, M., 2022. Moving towards model based approval – The open class 3D exchange (OCX) standard. Int. J. Marit. Eng. 164, 91–112. http://dx.doi.org/10.5750/ijme.v164iA2.1231.

Balse, R., Kumar, V., Prasad, P., Warriem, J.M., 2023. Evaluating the quality of LLM-generated explanations for logical errors in CS1 student programs. In: Proceedings of the 16th Annual ACM India Compute Conference. COMPUTE '23, Association for Computing Machinery, New York, NY, USA, pp. 49–54. http://dx.doi.org/10.1145/3627217.3627233.

Chen, R., Chen, C.-H., Liu, Y., Ye, X., 2020. Ontology-based requirement verification for complex systems. Adv. Eng. Inform. 46, 101148.

Cunha, A., Macedo, N., 2025. Validating formal specifications with LLM-generated test cases. arXiv preprint arXiv:2510.23350.

De Haan, P., Johnson, C., Efatmaneshnik, M., James, A., 2024. Using SysML to capture a naval ship's ontology for interdisciplinary communication. In: 2024 IEEE International Symposium on Systems Engineering. ISSE, IEEE, pp. 1–7.

Dimyadi, J., Clifton, C., Spearpoint, M., Amor, R., 2014. Regulatory knowledge encoding guidelines for automated compliance audit of building engineering design. In: Computing in Civil and Building Engineering (2014). pp. 536–543.

DNV, 2025. Maritime shipping regulation & compliance guide. https://www.dnv.com/maritime/decarbonize-shipping/regulations/. (Accessed 14 May 2026).

Ekaputra, F.J., Lin, X., 2016. SHACL4P: SHACL constraints validation within Protégé ontology editor. In: 2016 International Conference on Data and Software Engineering. ICoDSE, IEEE, pp. 1–6. http://dx.doi.org/10.1109/ICODSE.2016.7936162.

Elgammal, A., Turetken, O., van den Heuvel, W.-J., Papazoglou, M., 2016. Formalizing and appling compliance patterns for business process compliance. Softw. Syst. Model. 15, 119–146.

European Parliament and Council, 2015. Regulation (EU) 2015/757 on the monitoring, reporting and verification of carbon dioxide emissions from maritime transport. Off. J. Eur. Union L 123, 55–76.

European Parliament and Council, 2023a. Directive (EU) 2023/959 amending directive 2003/87/EC establishing a system for greenhouse gas emission allowance trading within the Union. Off. J. Eur. Union L 130, 134–202, Extends EU ETS to maritime transport.

European Parliament and Council, 2023b. Regulation (EU) 2023/1805 on the use of renewable and low-carbon fuels in maritime transport, and amending directive 2009/16/EC. Off. J. Eur. Union L 234, 48–100.

Faria, J.P., Trigo, E., Honorato, V., Abreu, R., 2026. Automatic generation of formal specification and verification annotations using LLMs and test oracles. arXiv preprint arXiv:2601.12845.

Finnish Board of Navigation, 1985. Board of navigation rules for assigning ice classes to ships. Historical baseline for hull structural rules. Board of Navigation Circular No. 11/1985.

Finnish Transport and Communications Agency (Traficom), 2021. Regulation on the ice classes of ships and icebreaker assistance. TRAFICOM/68863/03.04.01.00/2021. https://www.traficom.fi.

Finnish Transport and Communications Agency (Traficom), 2024. Regulation on finnish ice classes equivalent to class notations of recognized organizations. Entered into force 1 January 2025.

Finnish Transport Safety Agency (Trafi), 2010. The structural design and engine output required of ships for navigation in ice: Finnish-Swedish ice class rules. Tech. Rep. TRAFI/31298/03.04.01.00/2010 Finnish Transport Safety Agency, Helsinki, Finland, URL https://www.sjofartsverket.se/globalassets/tjanster/isbrytning/pdf-regelverk/finnish-swedish_iceclass_rules.pdf.

Gearon, P., Passant, A., Polleres, A., 2013. SPARQL 1.1 Update. Tech. rep. W3C, URL https://www.w3.org/TR/sparql11-update/. W3C Recommendation.

Gholami, A., Jazayeri, S.A., Esmaili, Q., 2020. Marine powertrain simulation for design and operational performance evaluation. Int. J. Powertrains 9 (4), 289–314.

Gourdon, K., 2019. An analysis of market-distorting factors in shipbuilding. OECD Science, Technology and Industry Policy Papers.

Graves, H., 2009. Integrating sysml and OWL.

Harder, B., Esser, S., Borrmann, A., 2025. Interpreting SysML diagrams in conjunction with OWL ontologies for semantic reasoning and inference.

Hassani, S., 2024. Enhancing legal compliance and regulation analysis with large language models. In: 2024 IEEE 32nd International Requirements Engineering Conference. RE, IEEE, pp. 507–511.

Heimsbakk, V., Torkelsen, K., 2023. Using the shapes constraint language for modelling regulatory requirements. arXiv preprint arXiv:2309.02723.

Herber, D.R., Narsinghani, J.B., Eftekhari-Shahroudi, K., 2022. Model-based structured requirements in SysML. In: 2022 IEEE International Systems Conference. SysCon, IEEE, pp. 1–8.

International Maritime Organization, 2021. Resolution MEPC.328(76): Amendments to MARPOL annex VI (2021 revised MARPOL annex VI). Introduces EEXI and CII, entered into force 1 November 2022.

Jansen, N., Pfeiffer, J., Rumpe, B., Schmalzing, D., Wortmann, A., 2022. The language of SysML v2 under the magnifying glass. J. Object Technol. 21 (3).

Ke, J., Zacouris, Z., Acosta, M., 2024. Efficient validation of SHACL shapes with reasoning. Proc. VLDB Endow. 17 (11), 3589–3601.

Knublauch, H., Bergwinkl, T., Taghzouti, Y., Wright, J., 2026. SHACL 1.2 core. World Wide Web Consortium (W3C). URL https://www.w3.org/TR/2026/WD-shacl12-core-20260105/. (Accessed 10 March 2026).

Knublauch, H., Maria, P., 2017. SHACL JavaScript extensions. World Wide Web Consortium (W3C). URL https://www.w3.org/TR/shacl-js/. (Accessed 10 March 2026).

Lee, S.W., Gandhi, R.A., 2005. Ontology-based active requirements engineering framework. In: 12th Asia-Pacific Software Engineering Conference. APSEC'05, IEEE, pp. 8–pp.

Leinonen, J., Hellas, A., Sarsa, S., Reeves, B., Denny, P., Prather, J., Becker, B.A., 2023. Using large language models to enhance programming error messages. In: Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1. pp. 563–569.

Liu, D., Cheng, L., 2024. MAKG: A maritime accident knowledge graph for intelligent accident analysis and management. Ocean Eng. 312, 119280.

Lloyd's Register, 2025. New report simplifies complex regulatory landscape. https://www.lr.org/en/knowledge/insights-articles/new-report-simplifies-complex-regulatory-landscape/. (Accessed 14 May 2026).

Minker, J., 1982. On indefinite databases and the closed world assumption. In: Loveland, D.W. (Ed.), 6th Conference on Automated Deduction. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 292–308.

Moore, P., Van Pham, H., 2015. On context and the open world assumption. In: 2015 IEEE 29th International Conference on Advanced Information Networking and Applications Workshops. IEEE, pp. 387–392.

Nuyts, E., Werbrouck, J., Verstraeten, R., Deprez, L., 2023. Validation of building models against legislation using SHACL. In: LDAC2023: Linked Data in Architecture and Construction Week, Vol. 3633, CEUR, pp. 164–175.

Ogbuji, C., 2013. SPARQL 1.1 Graph Store HTTP Protocol. Tech. rep. W3C, URL https://www.w3.org/TR/sparql11-http-rdf-update/. W3C Recommendation.

Olaniyi, E.O., Solarte-Vasquez, M.C., Inkinen, T., 2024. Smart regulations in maritime governance: Efficacy, gaps, and stakeholder perspectives. Marine Poll. Bull. 202, 116341.

Ritari, A., Huotari, J., Tammi, K., 2023. Marine vessel powertrain design optimization: Multiperiod modeling considering retrofits and alternative fuels. Proc. Inst. Mech. Eng. Part M: J. Eng. Marit. Environ. 237 (3), 597–614.

Sampson, H., Walters, D., James, P., Wadsworth, E., 2014. Making headway? Regulatory compliance in the shipping industry. Soc. Leg. Stud. 23 (3), 383–402.

Sannier, N., Baudry, B., Nguyen, T., 2011. Formalizing standards and regulations variability in longlife projects. a challenge for model-driven engineering. In: 2011 Model-Driven Requirements Engineering Workshop. IEEE, pp. 64–73.

Schmidt, R., Bartsch, C., Oberhauser, R., 2007. Ontology-based representation of compliance requirements for service processes. In: Proceedings of the Workshop on Semantic Business Process and Product Lifecycle Management (SBPM 2007), Held in Conjunction with the 3rd European Semantic Web Conference. ESWC 2007, pp. 28–39.

Serester, A., Hidas, G., Szögi, G., Galambos, P., 2015. Testing and verification through virtual product models: A survey and look ahead. In: 2015 IEEE 19th International Conference on Intelligent Engineering Systems. INES, IEEE, pp. 141–145.

Tadros, M., Ventura, M., Soares, C.G., 2023. Review of current regulations, available technologies, and future trends in the green shipping industry. Ocean Eng. 280, 114670.

TopQuadrant, 2017. SHACL API in JavaScript. URL https://github.com/TopQuadrant/shacl-js. GitHub repository.

Valdivieso, R.H., 2019. Regulatory safety considerations about ship design. Ship Sci. Technol. 12 (24), 9–16.

Wei, Q., Liu, Y., 2022. Ship design optimization framework considering future uncertain carbon emission regulations. In: SNAME International Marine Design Conference, http://dx.doi.org/10.5957/IMDC-2022-232, vol. Day 4 Wed, June 29, 2022. D041S013R001.

Zazuko, 2025. Rdf-validate-shacl: Validate RDF data purely in JavaScript. URL https://github.com/zazuko/rdf-validate-shacl. (Accessed 10 March 2026). GitHub repository.

Zhang, J., El-Gohary, N.M., 2015. Automated information transformation for automated regulatory compliance checking in construction. J. Comput. Civ. Eng. 29 (4), B4015001.

Zhang, J., El-Gohary, N.M., 2017. Semantic-based logic representation and reasoning for automated regulatory compliance checking. J. Comput. Civ. Eng. 31 (1), 04016037.

Zhang, K., Wang, D., Xia, J., Wang, W.Y., Li, L., 2023. Algo: Synthesizing algorithmic programs with generated oracle verifiers. Adv. Neural Inf. Process. Syst. 36, 54769–54784.

Zhong, B., Ding, L., Luo, H., Zhou, Y., Hu, Y., Hu, H., 2012. Ontology-based semantic modeling of regulation constraint for automated construction quality compliance checking. Autom. Constr. 28, 58–70.